# TEXT CLUSTERRING




**NAMA: TIARA ANANDA PARDOSI**

**NIM: 42324018**

**PRODI: D3 TEKNOLOGI INFORMASI**

**INSTITUT TEKNOLOGI DEL**

**FAKULTAS VOKASI**

**2026/2027**

## Konsep Text Clustering dengan K-Means

Text clustering adalah teknik untuk mengelompokkan sekumpulan dokumen teks ke dalam beberapa kelompok berdasarkan kemiripan isinya, tanpa label yang ditentukan sebelumnya. Dokumen yang membahas hal serupa akan masuk ke cluster yang sama, sedangkan dokumen yang topiknya berbeda akan terpisah.

Text clustering termasuk **unsupervised learning** (pembelajaran tanpa pengawasan). Berbeda dengan klasifikasi teks yang butuh data berlabel (misalnya "spam" atau "bukan spam"), clustering menemukan pola kelompok sendiri dari data. Pengguna hanya menentukan jumlah kelompok yang diinginkan, sedangkan arti tiap kelompok ditafsirkan sendiri setelah hasilnya keluar.

### Contoh pemanfaatan

- Mengelompokkan artikel berita berdasarkan topik.
- Mengelompokkan ulasan produk atau komentar pelanggan.
- Mengorganisasi dokumen atau email dalam jumlah besar.
- Sistem rekomendasi, misalnya merekomendasikan film dengan sinopsis yang mirip.
- Eksplorasi awal data teks sebelum analisis yang lebih dalam.

### Tahapan umum text clustering

| Tahap | Tujuan | Pada kode ini |
|---|---|---|
| Preprocessing | Membersihkan dan menyeragamkan teks | clean_text() dengan lowercase, hapus angka dan tanda baca, tokenisasi, lemmatisasi |
| Representasi teks | Mengubah teks menjadi vektor angka | TfidfVectorizer |
| Clustering | Mengelompokkan vektor yang mirip | KMeans |
| Visualisasi dan evaluasi | Melihat dan menilai hasil | PCA dan matplotlib |

## 3. Konsep Penting yang Dipakai

### 1. Preprocessing teks

Komputer tidak memahami teks mentah, dan teks mentah juga penuh variasi yang sebenarnya bermakna sama (misalnya "Movie", "movie", dan "movies"). Preprocessing menyeragamkan variasi tersebut agar hasil clustering lebih akurat. Langkah yang dipakai di kode ini:

- **Case folding**: mengubah semua huruf menjadi huruf kecil.
- **Penghapusan angka**: angka umumnya tidak membawa informasi topik.
- **Penghapusan tanda baca**: titik, koma, dan simbol lain dibuang.
- **Tokenisasi**: memecah kalimat menjadi daftar kata.
- **Lemmatisasi**: mengubah kata ke bentuk dasar kamus, misalnya "movies" menjadi "movie".
- **Penghapusan stop words**: kata umum seperti "the", "and", "is" dibuang. Pada kode ini langkah tersebut dilakukan oleh `TfidfVectorizer(stop_words='english')`.

### 2. TF-IDF

TF-IDF (Term Frequency, Inverse Document Frequency) adalah cara mengubah teks menjadi angka dengan memberi bobot pada setiap kata di setiap dokumen.

- **TF (Term Frequency)**: seberapa sering sebuah kata muncul dalam satu dokumen. Makin sering muncul, makin tinggi nilainya.
- **IDF (Inverse Document Frequency)**: seberapa langka kata itu di seluruh kumpulan dokumen. Kata yang muncul di hampir semua dokumen mendapat bobot kecil, sedangkan kata yang jarang muncul mendapat bobot besar.
- **TF-IDF = TF × IDF**.

Hasilnya, kata yang khas untuk suatu dokumen (misalnya "spaceship" atau "detective") mendapat bobot tinggi, sedangkan kata yang ada di mana-mana mendapat bobot rendah. Setiap dokumen menjadi sebuah vektor, dan banyaknya dimensi sama dengan banyaknya kata unik (kosakata). Karena kebanyakan kata tidak muncul di sebuah dokumen, vektornya berisi banyak nilai nol (sparse matrix).

### 3. K-Means

K-Means adalah algoritma clustering yang membagi data menjadi **K** kelompok. Cara kerjanya:

1. Tentukan jumlah cluster K (di kode ini K = 2).
2. Pilih K titik awal sebagai pusat cluster (centroid).
3. Tempatkan setiap data ke centroid terdekat (jarak Euclidean).
4. Hitung ulang posisi setiap centroid sebagai rata-rata data di dalam clusternya.
5. Ulangi langkah 3 dan 4 sampai posisi centroid tidak banyak berubah.

Hasil K-Means dipengaruhi oleh penempatan centroid awal. Karena itu kode memakai `random_state=42` agar hasil bisa diulang dan `n_init=10` agar algoritma dijalankan 10 kali dengan awalan berbeda, lalu diambil hasil terbaik.

**4. PCA**

PCA (Principal Component Analysis) adalah teknik reduksi dimensi. Vektor TF-IDF punya ribuan dimensi, sehingga tidak bisa digambar. PCA mencari arah-arah (komponen utama) yang menangkap variasi data paling besar, lalu memproyeksikan data ke 2 komponen pertama. Dengan begitu, setiap dokumen menjadi satu titik di grafik 2D.

Penting dipahami: PCA hanya dipakai untuk **visualisasi**. Clustering K-Means dilakukan pada data TF-IDF penuh, bukan pada hasil PCA.

**4. Penjelasan Kode per Bagian**

**1. Import library**

```
import pandas as pd
import numpy as np
import re
import string
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
✓ 3.0s
```

| Library | Fungsi |
|---|---|
| pandas | Membaca dataset dan mengelola data dalam bentuk tabel (DataFrame) |
| numpy | Operasi numerik dan array. Pada kode ini diimpor tetapi belum dipakai secara langsung |
| re | Regular expression, dipakai untuk menghapus angka |

| Library | Fungsi |
|---|---|
| string | Menyediakan daftar karakter tanda baca |
| word_tokenize (NLTK) | Memecah teks menjadi kata-kata |
| WordNetLemmatizer (NLTK) | Mengubah kata ke bentuk dasarnya |
| TfidfVectorizer | Mengubah teks menjadi vektor TF-IDF |
| KMeans | Algoritma clustering |
| PCA | Reduksi dimensi untuk visualisasi |
| matplotlib.pyplot | Membuat grafik |

Keterangan di bawah sel menunjukkan waktu eksekusi 3.0 detik. Perlu diingat bahwa NLTK membutuhkan resource punkt (atau punkt_tab) dan wordnet yang diunduh sekali lewat nltk.download(...) agar tokenisasi dan lemmatisasi bisa berjalan.

**2. Membaca dataset**

```
file_path = "tmdb_5000_movies.csv"
df = pd.read_csv(file_path)
df = df.rename(columns={'overview': 'text'})
✓ 0.1s
```

- file_path menyimpan nama file dataset, yaitu TMDB 5000 Movies yang berisi data sekitar 5000 film.
- pd.read_csv(file_path) membaca file CSV menjadi DataFrame bernama df.
- df.rename(columns={'overview': 'text'}) mengganti nama kolom overview (sinopsis film) menjadi text. Sejak baris ini, kolom teks yang dipakai di seluruh kode bernama text.

Kolom overview dipilih karena berisi deskripsi cerita film dalam bahasa Inggris, sehingga cocok sebagai bahan clustering teks.

**3. Inisialisasi lemmatizer dan fungsi preprocessing**

```
lemmatizer = WordNetLemmatizer()

Tabnine | Edit | Test | Explain | Document
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'\d+', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    words = word_tokenize(text)
    words = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(words)
```

Baris lemmatizer = WordNetLemmatizer() membuat objek lemmatizer. Tanda kurung wajib ada karena objek harus dibuat dari kelas tersebut. Selanjutnya fungsi clean_text memproses satu teks melalui enam langkah:

1. str(text).lower(): memastikan data berupa string, lalu mengubahnya menjadi huruf kecil. Pemanggilan str() mencegah error bila ada nilai yang bukan teks.
2. re.sub(r'\d+', '', text): menghapus semua angka. Pola \d+ berarti satu atau lebih digit berurutan, dan diganti dengan string kosong.
3. text.translate(str.maketrans('', '', string.punctuation)): menghapus semua tanda baca. Argumen ketiga pada maketrans berisi karakter yang akan dihapus.
4. word_tokenize(text): memecah teks menjadi daftar kata.
5. [lemmatizer.lemmatize(word) for word in words]: mengubah setiap kata ke bentuk dasarnya dengan list comprehension. Secara bawaan lemmatizer memperlakukan kata sebagai kata benda, sehingga "movies" menjadi "movie", tetapi kata kerja seperti "running" tidak selalu berubah.
6. ' '.join(words): menggabungkan kembali kata-kata menjadi satu kalimat dengan pemisah spasi. Pemisah spasi penting agar kata tidak menyatu.

Contoh: kalimat "In 2154, the Movies are Running!" menjadi kira-kira "in the movie are running".

**4. Membersihkan data kosong dan menerapkan preprocessing**

```
df = df.dropna(subset=['text']).copy()
df['clean_text'] = df['text'].apply(clean_text)
```

- df.dropna(subset=['text']) membuang baris yang kolom text-nya kosong (NaN). Sinopsis kosong tidak bisa diproses dan tidak berguna untuk clustering.
- .copy() membuat salinan DataFrame agar tidak muncul peringatan pandas (SettingWithCopyWarning) saat kolom baru ditambahkan.
- df['text'].apply(clean_text) menjalankan fungsi clean_text pada setiap baris, lalu hasilnya disimpan di kolom baru clean_text. Kolom text yang asli tetap utuh.

**5. Representasi TF-IDF**

```
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(df['clean_text'])
```

- TfidfVectorizer(stop_words='english') membuat objek TF-IDF yang otomatis membuang stop words bahasa Inggris.
- fit_transform bekerja dalam dua tahap sekaligus. Tahap *fit* mempelajari kosakata dan nilai IDF dari seluruh dokumen. Tahap *transform* mengubah tiap dokumen menjadi vektor angka.
- X adalah sparse matrix berukuran (jumlah film) × (jumlah kata unik). Setiap baris mewakili satu sinopsis dan setiap kolom mewakili satu kata.

**6. Clustering dengan K-Means**

```
num_clusters = 2
kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X)
```

- num_clusters = 2 menentukan jumlah kelompok yang diinginkan, yaitu 2.
- n_clusters adalah jumlah cluster, random_state=42 mengunci keacakan agar hasil konsisten setiap dijalankan, dan n_init=10 menjalankan K-Means 10 kali dengan centroid awal berbeda lalu memilih hasil terbaik.
- fit_predict(X) melatih model sekaligus memberi label cluster untuk setiap dokumen. Hasilnya berupa angka 0 atau 1 yang disimpan di kolom baru cluster.

Angka 0 dan 1 hanyalah penanda kelompok, bukan skor atau peringkat. Arti tiap cluster harus ditafsirkan sendiri, misalnya dengan melihat kata-kata dominan di masing-masing kelompok.

**7. Reduksi dimensi dengan PCA**

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X.toarray())
```

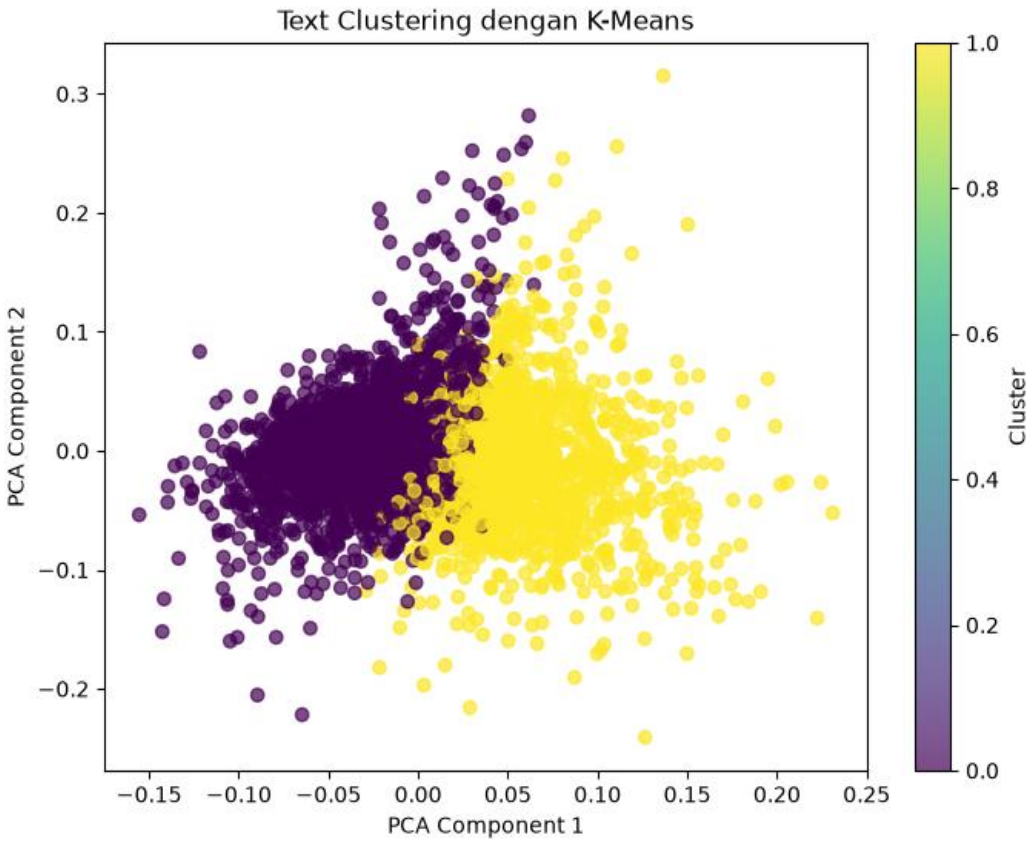
- PCA(n_components=2) mengatur PCA agar menghasilkan 2 komponen utama.
- X.toarray() mengubah sparse matrix menjadi array biasa karena PCA di scikit-learn membutuhkan data dalam bentuk padat. Langkah ini memakan memori besar bila datanya sangat banyak, tetapi masih wajar untuk sekitar 5000 dokumen.
- fit_transform menghasilkan X_pca, yaitu array dengan 2 kolom (koordinat X dan Y) untuk setiap dokumen.

**8. Visualisasi hasil**

```
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df['cluster'], cmap='viridis', alpha=0.7)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Text Clustering dengan K-Means")
plt.colorbar(label='Cluster')
plt.show()
```

- plt.figure(figsize=(8, 6)) membuat kanvas grafik berukuran 8 × 6 inci.
- plt.scatter(...) menggambar setiap dokumen sebagai satu titik. X_pca[:, 0] menjadi sumbu X (komponen 1), X_pca[:, 1] menjadi sumbu Y (komponen 2), c=df['cluster'] mewarnai titik sesuai cluster, cmap='viridis' memilih palet warna ungu ke kuning, dan alpha=0.7 membuat titik agak transparan sehingga area yang padat terlihat.
- plt.xlabel, plt.ylabel, dan plt.title memberi label sumbu dan judul.
- plt.colorbar(label='Cluster') menampilkan batang warna sebagai legenda.
- plt.show() menampilkan grafik.

## Interpretasi Output



Grafik hasil bernama "Text Clustering dengan K-Means" menampilkan ribuan titik, masing-masing mewakili satu sinopsis film, dengan dua warna:

- **Ungu (cluster 0)**: berkumpul padat di sisi kiri grafik, dengan pusat kepadatan di sekitar nilai PCA Component 1 negatif.
- **Kuning (cluster 1)**: berada lebih ke kanan dan sebarannya lebih lebar, dengan beberapa titik menjauh ke sisi kanan dan atas.

Beberapa hal yang bisa disimpulkan dari grafik:

1. **Pemisahan terutama terjadi pada sumbu PCA Component 1.** Kedua cluster terpisah secara garis besar dari kiri ke kanan.
2. **Ada area tumpang tindih di bagian tengah.** Ini wajar. Data teks berdimensi sangat tinggi, sedangkan grafik hanya menampilkan 2 komponen utama yang hanya menangkap sebagian kecil variasi data. Dua titik yang terlihat berdekatan di grafik belum tentu berdekatan di ruang TF-IDF yang sebenarnya, dan sebaliknya.
3. **Tidak ada pemisahan yang tegas.** Sinopsis film mencakup tema yang saling beririsan (misalnya aksi, drama, dan romansa dalam satu cerita), sehingga batas antartopik tidak tajam.
4. **Batang warna menampilkan skala 0 sampai 1** hanya karena `colorbar` memetakan label cluster ke gradasi warna. Nilai yang bermakna hanya 0 dan 1.

Untuk mengetahui makna tiap cluster, langkah lanjutan yang bisa dilakukan adalah menampilkan kata dengan bobot TF-IDF tertinggi pada masing-masing centroid, atau membaca contoh sinopsis dari tiap cluster.